

es confused with Continuous Deployment. Continuous Deployment means that every change goes through the pipeline and automatically gets put into production, resulting in many production deployments every day. Continuous Delivery just means that you are able to do frequent deployments but may choose not to do it, usually due to businesses preferring a slower rate of deployment. In order to do Continuous Deployment you must be doing Continuous Delivery.
> \- Martin Fowler on continuous delivery, 2013

What

- Can mean either Continuous Delivery and/or Deployment
- The main difference between continuous delivery and deployment is delivery = manual release, deployment = automated release
- Either way, CD is a release management practice
- Continuous Delivery means that the software is always ready to be released
 - Can mean it is ready to be pushed to a staging or production system
 - Or that it is ready to be shipped/distributed (not everyone does web apps)
- Continuous Deployment means that changes are actually released to production automatically when the pipeline successfully completes
- An automated, reliable and repeatable deployment process is critical for CD
- An immutable way to store and reference your deployment artifacts is critical for CD
 - eg. Artifact repository, Container registry
- Feature flags are important to enable CD with minimal impact to customers
- Deployment visibility is important with CD to keep track of what is where with the increased deployment frequency

Why

- Continual testing for releasability lowers the risk of each release - goal: boring releases
- When releases are low risk, they happen more, and value gets delivered to customers more frequently

Who

- All software organizations (at least for "continuous delivery")
- Not all software organizations need to, want to, or can get to "continuous deployment"
- If an org has a DevOps initiative, even if not explicitly defined, DevOps serves to enable CD

Market Overview

Forrester CI Wave

- No vendor can survive today by only doing CI
- Vendors in the CI space have all evolved to include CD, as it is a natural extension of the capabilities of a CI system
- This evolution largely involves adding integrations to provision and configure environments (the automation is already a capability)
- Containers, micro-services, and Kubernetes changed the game of what it takes to do CI/CD
 - build artifacts through the pipeline now come wrapped in their application environments

- storing container images instead of build artifacts
- full integration testing becomes easier in earlier stages
- CI/CD tools can now more easily automate everything about an applications deployment with minimal integration work
- So now CI/CD vendors are all racing to add Docker and Kubernetes support

Competitor Scope - Continuous Integration

Feature	GitLab	CircleCI	Azure DevOps	Jenkins	Travis CI	Bitbucket
Auto-scaling	Y	Y	N	Integration	Y	Y
Pipeline graphs	Y	Y	Y	Integration	N	
Integration						
Parallel jobs	Y	Y	Y	Y	Y	Y
Config as Code	Y	Y	Y	Y	Y	Y
Native Docker support	Y	Y	Y	Integration	Y	Y

Competitor Scope - Continuous Delivery/Deployment (CI products)

Feature	GitLab	CircleCI	Azure DevOps	Jenkins	Travis CI	Bitbucket
Environments	Y	N	Y	N	Y	Y
Env History	Y	N	N	N	N	Y
No Conf staging apps	Y	N	N	N	Y	N
Deploy boards	Y	N	Y	Integration	Y	N
Native Canary Deploy	Y	N	Y	N	Y	N
Secure variables	Y	N	Y	Integration	N	Y
Deploy tracability	Y	N	Y	Integration	Y	Y
Protected Envs	Y	N	Y	N	N	Y
Native Kubernetes support	Y	N	Y	Integration	N	N
Native Feature flags	Y	N	N	N	N	N

Competitor Scope - Continuous Delivery/Deployment (CD products)

Feature	GitLab	Spinnaker	Azure DevOps	XebiaLabs	ElectricFlow	CodeFresh
Environments	Y	I-	Y	I-	Y	I-
Env History	Y	I-	N	I-	Y	I-
No Conf staging apps	Y	I-	N	I-	N	I-
Deploy boards	Y	I-	Y	I-	Y	I-
Native Canary Deploy	Y	I-	Y	I-	Y	I-
Secure variables	Y	I-	Y	I-	Y	I-
Deploy tracability	Y	I-	Y	I-	Y	I-
Protected Envs	Y	I-	Y	I-	Y	I-
Native Kubernetes support	Y	I-	Y	I-	N	I-
Native Feature flags	Y	I-	N	I-	N	I-

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/core-product-marketing/_index.md

Product marketing at GitLab

Product Marketing communicates GitLab business value internally and externally to position GitLab as a DevOps partner and solution. The team is responsible for product positioning, value messaging, and go-to-market strategy to support sales and outbound messaging with analysts, press, and prospects. Product Marketing also facilitates market feedback as key inputs into the GitLab product roadmap.

PMM Stable Counterparts

- The PMM Team
- PM:PMM stable counterparts are listed on the Product Sections page.
- Solution DRIs are listed on the Solutions GTM page.

Core Competencies

PMMs require a broad set of skills, knowledge, and expertise. This lists core competencies.

- Narrative, Messaging, and Positioning
- Writing/editing
- End-to-end buyers journey
 - All stages of the marketing funnel - TOFU, MOFU, BOFU
 - All stages of the sales cycle
 - Customer marketing
- Product knowledge
- Market knowledge
 - Competitors
 - Partners
 - Peers (Other tech products and companies that aren't partners or competitors, but exist as part of the ecosystem)
 - Trends
- Buyer and user personas

Onboarding

- **Go to the PMM onboarding page** and check out our onboarding guide.

Product Marketing Collaboration

In general, product marketing supports three main groups:

1. Enables sales with positioning, messaging, content, and collateral.
 1. Create decks, messaging, and collateral
 1. Support and deliver sales enablement
 1. Actively engage in the sales cycle
1. Supports campaigns, content, and field marketing teams with content, webinars, presentations, and strategic input.
 1. Drafts content, whitepapers, webinars, and presentations
1. Partners with the Product teams on vision, roadmaps, and new features to improve our differentiation.
 1. Feedback and input into vision and roadmap
 1. Drafting the Release Post Headline and Introduction
 1. Reviewing and updating feature descriptions

Sales Collaboration

In addition to enabling sales, PMMs should partner with sales and actively engage with the sales process by listening to call recordings and joining customer calls with reps. There are several outcomes that result from this process:

1. Improved Messaging - test messaging directly on end customers to gauge their reaction and get feedback. Understand their pain points and values in order to use them to fuel your messaging.
2. Know the buyers journey end-to-end - understand the stages of the buyer process by actively engaging with them.
3. Assist sales on key deals - PMM can serve as product and market SMEs who can tell the story of their solutions in a compelling way to help move key deals forward.
4. PM triage/assist/feedback -As part of the PM <> PMM partnership PMMs can recommend customers

they've talked to their PMs to engage with and assist in the gathering of product feedback during customer calls to share with the PM. Similarly, PMs can share messaging feedback, buyer persona info, and sales process insights they gather on calls with customers with their PMMs.

Metrics and Measures

1. ARR per solution: Product Marketing Managers are the "Product Managers of Marketing". In the same way a Product Manager is responsible for the product research & development, the Product Marketing Manager is responsible for the product go-to-market. In the same way PMs don't have direct authority over engineering, but they collaborate and lead by influence, PMMs don't have direct authority over sales and marketing execution but they collaborate and lead by influence.

- At GitLab, PMs are measured on usage of their product area even though they don't have complete control over all of the factors that drive usage, it is a good proxy to help them make decisions that best impact the user.

- At GitLab, PMMs are measured on ARR of their solution(s) even though they don't have complete control over all of the factors that drive ARR, it is a good proxy to help them make decisions that best impact the buyer at every stage of the funnel/buyer journey.

Solutions Go To Market

Solutions GTM Page

Project Management

1. Every project should have a top-level Epic, Issue, doc that outlines the plan and links to all supporting issues and docs.

1. The plan chart should include columns for

- 1. Deliverable or Milestone

- 1. DRI

- 1. Due Date is ISO format (point 7)

- 1. Status: On track, Needs Attention, At risk, or DONE

- 1. Note/Next step (optional)

1. You can use a google sheet, markdown table (in an issue or epic description), or Epic issue list for the plan

1. If you use an Epic issue list follow these constraints:

- 1. Every deliverable should have an issue attached to the epic.

- 1. The issue title must be the name of the deliverable .

- 1. The deliverable draft, and when completed final version, should be linked at the top of the issue description.

- 1. Every issue should have only 1 assignee - the DRI for the deliverable.

- 1. Every issue should have a due date.

- 1. Every issue should have a health status that is checked/updated weekly. (Close the issue for deliverables that are complete/done.)

Health Status Definitions

- On Track: We are confident this issue will be completed by the due date.

- Needs Attention: There are concerns, new complexity, or unanswered questions that if left unattended will result in the issue missing its targeted completion date. Collaboration needed to get back On Track.

- At Risk: The issue in its current state will not make the targeted completion date and immediate action is needed to rectify the situation.
- DONE: The deliverable is completed and linked. A closed issue represents a status of "DONE".

Key links

- Messaging
- GitLab positioning
- Hidden IT Groups
- Defining GitLab roles and personas
- GitLab tiers
- Market segmentation - Industry verticals
- Use Case - Go to market

PMM Talk Inventory

```
<iframe width="950" height="900" src="https://docs.google.com/spreadsheets/d/e/2PACX-1vSvkbjALQnaEuvG0iUEfFStZttVxMP0WCzTaDIGS31ZLndVvBgaEWnOkrRYRPYQtj6g9bJV2J0C/pubhtml?single=true&widget=true&headers=false&rm=minimal&range=publishedtalks"></iframe>
```

Release vs. launch

A product release, and a marketing launch are two separate activities. The canonical example of this is Apple. They launch the iPhone at their yearly event and then release it months later. At GitLab we do it the other way: Release features as soon as they are ready letting customers use them right away, and then, do a marketing launch later when we have market validation.

Release	Launch
PM Led	PMM Led
New features can ship with or without marketing support	Launch timing need not be tied to the proximity of when a feature was released

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/core-product-marketing/onboarding.md

Onboarding Guide

Welcome to the Product Marketing team! Use the resources/videos on this page as a guide to help facilitate new team member onboarding. As more recording links and resources become available, we'll update this page accordingly.

Each topic covered in the tables below should be presented in 3-5 minutes or less through the handbook/screen share and recorded. Exceptions: Vision, Pitch Deck, etc. The goal of the completion time for PMM onboarding specific tasks is 1 day (except coffee chats).

- Recording folder

Background

Intro to Git

What is Git - A video tutorial by @gitschooldude	Entire Playlist
1. Introduction to Git	Sept 2015
2. Your first Git repository	Sept 2015
3. Introduction to Git Push	Oct 2015
4. Introduction to Git Branches	Nov 2015

Intro to GitLab

What is GitLab - A video tutorial by @gitschooldude	Entire Playlist
1. Introduction to GitLab	May 2017
2. Introduction to GitLab CI	April 2018
3. GitLab CI's vision: artifacts, workspaces, and runners	Dec 2018
4. Introduction to Docker and GitLab Registry PART (1/2)	Jan 2019
5. Introduction to Docker and GitLab Registry PART (2/2)	Jan 2019
6. Parallel jobs in GitLab CI	Feb 2019
7. GitLab Merge Request (MR) merge methods	March 2019
8. GitLab CI's biggest design flaw	April 2019
9. GitLab CI advanced job triggers: branch names, commit messages, and changed files	
June 2019 |

Coffee Chats

Note: Remember to complete your personalized company onboarding agenda.
Pay special attention to the coffee chats. Typical onboarding suggests new team members to schedule at least 5 coffee chats. Because PMMs must work cross-functionally, relationships and connections are critical for success. We suggest scheduling ~20 coffee chats over 6 weeks. Suggested list (Talk with your manager to pick names)

Process

PMM Specific Processes

PMM Workflow:	Recording
1. Everything starts with the handbook - Product Marketing handbook	Recording
2. Use issues to drive your work - Product Marketing board	Recording
3. Everything you create/update for the field should be linked from - Sales resource page	
Recording |

Working Efficiently

Productivity:	Recordings
1. GitLab communication overview	tbd

- | 2. GitLab Slack tips | tbd |
- | 3. Markdown 101 | Recording |
- | 4. GitLab project management | Recording |
- | 5. Working Remotely to update the website with Git | Recording |

Messaging and Go-To-Market (GTM)

Learn the Marketing / Product Marketing Vision

- Slides
- Recording

- | Know the GitLab Messaging and GTM: | Recordings |
- | ----- | ----- |
- | 1. Know the GitLab value proposition - Company elevator pitch(es) | Recording |
- | 2. Company pitch deck | Recording |
- | 3. Security pitch <!-- William: for PMMs we should have a comprehensive "this is the product knowledge you should come up to speed on" · Ashish: Yes, eventually 1 list for each function. --> | tbd <!-- @cblake - let's discuss when --> |
- | 4. Walkthrough of stages on GitLab home page . | tbd |
- | 5. Explore our click-through demos | tbd |
- | 6. How to navigate our Product vision and roadmap webpages | tbd |

Technical Skills

Advanced Topics

- | Technical know-how: | Recordings |
- | ----- | ----- |
- | 1. Learn Git: Code Academy & Learning Git Branching | tbd |
- | 1a. Git advanced topics - making peace with merge conflicts | tbd |
- | 2. Learn CI/CD | tbd |
- | 3. Kubernetes (K8s) 101 from the Cloud Native Computing Foundation (CNCF)| No need for video, just checkout the pdf |
- | 4. How to create Kubernetes cluster - step by step guide | tbd |
- | 5. GitLab CI 101 | tbd |

The Who

Learn About Our Partners

- | Get to know our partners | Recordings |
- | ----- | ----- |
- | 1. Technology Partners overview | tbd |
- | 2. Channel Partners and Resellers overview | tbd |

Learn About Our Customers

- | Get to know our customers: | Recordings |
- | ----- | ----- |

- | 1. Customer Reference Program overview | Recording |
- | 2. Customer Advisory Board overview | Recording |

Learn About The Competition

- | Get to know the competition: | Recordings |
- | ----- | ----- |
- | 1. Walkthrough of DevOps tools landscape page | Recording |

Analyst Relations

Overview

- | Analyst Relations: | Recordings |
- | ----- | ----- |
- | 1. How we work the process? | tbd |
- | 2. How to work with analysts? | tbd |
- | 3. What reports are we in today? | tbd |

Useful links / resources

1. GitLab Team members
2. GitLab team structure

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/customer-advocacy/_index.md

Overview:

The Customer Advocacy team fosters evergreen co-marketing relationships with customer accounts and advocates that fuel an integrated strategy aligned to pipeline-driving initiatives, threading the customer voice into GTM motions.

- We create programs, content, and opportunities that put enterprise logos and advocates in the spotlight.
- We partner with Sales and Customer Success to identify, capture, and tell the right stories that help us sell and reinforce our enterprise-trusted brand promise.
- We support the bottom line by fueling our integrated campaign motion with powerful customer evidence.

Meet the team:

- Nicole Smith · Director, Product Marketing, Customer Advocacy
- Jocelyn Hernández Vázquez · Product Marketing Manager, Customer Advocacy
- Rachel Lurie · Product Marketing Manager, Customer Advocacy

How to work with us:

- Questions - Reach us in the customer-advocacy-and-references Slack channel.
- Information & Resources - Review the Customer Advocacy Overview deck and the Customer Stories Hub in Highspot.
- Requests - To create a general request for the Customer Advocacy team, open an issue [here](#).

Programs we drive and support

Customer Stories

The Customer Advocacy team partners with Content, Brand, and Design to run a continual, evergreen pipeline of customer stories in various formats including case studies, blogs, and marquee videos. We spotlight customer achievements and the DevSecOps teams that drive their success. The stories we publish are aligned to strategic pillars that support sales needs and integrated initiatives.

We strive to produce three customer stories per quarter and two marquee videos per year. A selection and prioritization process helps us tell the right stories in the right format. Selection criteria is based on GTM goals determined through cross-functional collaboration and strategic alignment, with additional prioritization based on account size, license type and number of users, and use cases.

> Have a customer with a compelling story? Help us spotlight your customer by completing the Customer Success Story Idea Submission Form.

Where to find customer stories:

- Customer Stories Hub in Highspot - filter by industry, region, ROI, and more; create a 'pitch' and share out your content of choice directly from Highspot.
- Customer Advocacy & Case Study Slides - position the value of participating in advocacy activities to your customers, and use the case study slides in presentations/pitches.
- Customer Case Studies (website)
- Customer Blogs (website)
- Customer Videos (YouTube playlist)

Earnings Customer References

The Customer Advocacy team partners with Investor Relations, Sales, and PMM to identify and showcase customer stories and name drops on our earnings calls each quarter.

We focus on two categories of earnings references:

- First Order: net new ENT/MM customers typically appear in earnings as name and/or logo drops.
- Growth: existing ENT/MM customers that have seen measurable success or expansion are typically featured with a story or quote in earnings.

> Help us secure earnings customer references. For more details on this process, please review the Customer References Sales Guide.

Customer Logo & Name Usage · Referenceable Customers

In addition to earnings calls, the Customer Advocacy team manages all approved customer references.

Customer reference types vary, and include all approved customer names, logos and pieces of customer evidence that can be leveraged in varied formats publicly in market. These may appear as long-form stories like case studies, blogs, and videos, short proof points like earnings mentions, quotes, and ROI metrics, or simple name and logo placements on GTM materials.

The team continuously updates the Customer Advocacy SSOT and the Customer ROI Evidence House. Leverage these trackers to find a name/logo you're interested in using, and work with the Customer Advocacy team for final review and approval.

> Have a customer that could be a reference? References should be secured in writing - ideally in the contract/order form, in the standalone customer license agreement, or via email. For more information, see the Customer References Sales Guide.

Speaking Opportunities

The Customer Advocacy team provides support for select GitLab hosted and sponsored events by sourcing customer speakers to share about their journeys, successes, and learnings in partnership with GitLab.

To request a customer to speak at and/or otherwise support an event, please open the Customer Speaker Request issue template a minimum of 60 days before the event and assign it to Jocelyn Hernández Vázquez.

Press Opportunities

Customer Advocacy partners with the Corporate Communications team to amplify the voice of customers through various media opportunities such as customer quotes in press releases and media interviews. We strive to pitch press opportunities that will provide the most value to the customer and allow for a continued warm relationship.

To secure a customer quote or an advocate for a media interview, please tag Nicole Smith in the issue at least 30 days prior to the deadline.

Peer Reviews & Analyst Interviews

GitLab maintains its leadership with Gartner Magic Quadrant, Forrester Wave, and IDC Forecasts by securing customers to participate in analyst interviews and market research surveys each quarter.

The Analyst Relations team aligns with the Customer Advocacy team on new survey or interview needs as early as possible to ensure ample time to source the required customer participants.

Executive Advisory Board (EAB) Program

The EAB is an invitation-only cohort of 12-15 VP+ level executives from select GitLab enterprise customer accounts who are driving business and technology transformation with a visionary DevSecOps strategy. Throughout the annual program, the board meets both in person and

virtually to engage with GitLab leaders, learn innovative best practices, and share insights to help shape the future of GitLab. For more details on the EAB program, review the Internal Brief.

- > Other advisory and executive customer programs managed by other teams at GitLab include:
 - >
 - > - Product Customer Advisory Board - Select manager+ level champions from strategic accounts, focused on technical discussions (DRI: Michaela Seferian-Jenkins, Product)
 - > - Federal Customer Advisory Board - Invitation-only cohort of executives from PubSec customer accounts (Interim DRI: Liz Burrows, PMM PubSec)
 - > - Federal Advisory Board - Forum of 5-7 paid consultants across the government (Interim DRI: Liz Burrows, PMM PubSec)
 - > - Executive Sponsorship Program - Pairs strategic customer executives with GitLab senior leadership for regular 1x1 engagement (Interim DRI: Aileen Lu, Sales)

Customer Reference Sales Contests

On an adhoc basis, the Customer Advocacy team runs contests to incentivize the Sales team to identify and secure customer references. The goal is to uncover new, high quality customer references whose names, logos, and stories can help strengthen our standing in the market.

Contests typically run throughout a designated quarter and offer a reward structure where account owners can qualify for monetary incentives by securing customer reference agreements.

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/demo/_index.md

About Demos

{{% alert title="Note" color="danger" %}}

THIS PAGE HAS BEEN DEPRECATED. IT IS STILL HERE DURING A TRANSITION PERIOD WHILE WE ALL THE CONTENT IS AVAILABLE ELSEWHERE

{{% /alert %}}

The new version of this page is at <https://university.gitlab.com/>

There are 3 basic demos types:

- Vision - designed to show where we are going (this might include forward looking ideas not yet developed)
- Use Case Based - designed to show how GitLab solves a particular use case (how we should be selling)
- Feature - designed to enable a deeper dive into details of product features (on prospect request)

These demos come in different formats:

- Videos - good for everyone to self-demo by passively watching
- Sim demos - good for disconnected situations (like conferences) and for interactive self

demos

- Live (instructions for how to set up and run demos live) - enables everyone (especially GitLab sales) to set up and run through the demo live on their own system resources. Allows for spontaneous deep diving - not always a good thing).

Videos

Foster Collaboration with GitOps (GitLab 13.2) (July 2020)

```
<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/onFpjwvblM" frameborder="0"
allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
</figure>
```

Version Control and Collaboration - Control Changes to Product Development Assets (April 2020)

```
<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/l6K3Xn2MPJw"
frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
</figure>
```

Version Control and Collaboration - Manage, Track and Maintain Access (April 2020)

```
<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/nRxCz4vMv5Q"
frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
</figure>
```

Version Control and Collaboration - Foster Collaboration (April 2020)

```
<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/OFNUjvgm24" frameborder="0"
allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
</figure>
```

GitLab CI - Visibility and Collaboration (April 2020)

```
<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/z8r3rFQT8xg"
frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
</figure>
```

DevSecOps - Managing Security Vulnerabilities with the Security Dashboard (April 2020)

```
<figure class="videocontainer">
```

```
<iframe width="560" height="315" src="https://www.youtube.com/embed/t-3TSIChHy4"
frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
</figure>
```

DevSecOps - Adding Security to the CI/CD pipeline (April 2020)

```
<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/Fd5DhebtScg"
frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
</figure>
```

DevSecOps - Dependency Scanning (April 2020)

```
<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/39RvTMLDszc"
frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
</figure>
```

DevSecOps - Container Scanning (April 2020)

```
<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/wlcaSerMfFQ"
frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
</figure>
```

DevSecOps - License Compliance (April 2020)

```
<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/42f9LiP5J4" frameborder="0"
allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
</figure>
```

Getting started with GitLab (12.9) (April 2020)

```
<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/0GFEV0rAC0" frameborder="0"
allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
</figure>
```

Ease of CI configuration (12.9) (April 2020)

```
<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/DGWqJotKQyM"
frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
```

allowfullscreen></iframe>
</figure>

Build and test automation (12.8) (March 2020)

<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/6207TKNGgJs"
frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
</figure>

DevSecOps Use Case Overview (12.7) (Feb 2020)

<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/UgCHtr-6uG8"
frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
</figure>

Benefits of a Single App (12.1) (Aug 2019)

<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/MNxkyLrA5Aw"
frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
</figure>

GitLab in 3 minutes (12.1) (Aug 2019)

<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/Jve98tIZ394"
frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
</figure>

GitLab Overview - Planning to Monitoring (11.3) (Oct 2018) (18 mins)

<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/nMAgP4Wlcno"
frameborder="0" allow="autoplay; encrypted-media" allowfullscreen></iframe>
</figure>

Set up groups and projects to run multiple Agile teams with microservices (11.7) (Jan 2019)

<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/VR2r1TJCDew"
frameborder="0" allow="autoplay; encrypted-media" allowfullscreen></iframe>
</figure>

SAFe and Agile Planning with GitLab (11.7) (Jan 2019)

<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/PmFFITH2DQk"
frameborder="0" allow="autoplay; encrypted-media" allowfullscreen></iframe>
</figure>

2018 Vision Prototype (Feb 2018)

<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/RmSTLGnEmpQ"
frameborder="0" allow="autoplay; encrypted-media" allowfullscreen></iframe>
</figure>

Auto DevOps in GitLab 11.0 (June 2018)

<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/OTc0YYBxqi4"
frameborder="0" allow="autoplay; encrypted-media" allowfullscreen></iframe>
</figure>

GitLab VSM - Issue Boards for Mapping (June 2018)

<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/9ASHiQ2juYY"
frameborder="0" allow="autoplay; encrypted-media" allowfullscreen></iframe>
</figure>

GitLab VSM - Business Value Monitoring (June 2018)

<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/oG0VESUOFAI"
frameborder="0" allow="autoplay; encrypted-media" allowfullscreen></iframe>
</figure>

GitLab for the Enterprise (Feb 2018)

<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/gcWfUwCau4"
frameborder="0" allow="autoplay; encrypted-media" allowfullscreen></iframe>
</figure>

Click-throughs

Click-through demos can be run in multiple ways:

- Run them off-line (for conferences, practice on the airplane, etc)
- Run them while online, from the files, same as running a presentation
- Or run them online from the web, using the links bellow:

GitLab Planning to Monitoring (Aug 2019 - 12.0)

<!--T Downloadable macOS

Downloadable Windows-->

Create Kubernetes cluster (Aug 2019 - 12.0)

<!--Downloadable macOS

Downloadable Windows-->

Merge Request Approval API (July 2019 - 12.0)

<!--Downloadable macOS

Downloadable Windows-->

Agile Project Management (June 2019 - 12.0)

<iframe src="https://docs.google.com/presentation/d/e/2PACX-1vR5EWjzgr-Qe-cYdMj4WJnnVDxKyoyirMv0OfOJWbgCzevJoLGTXnbKDYm2TUhpdpAkh-nTcucIgZKx/embed?start=false&loop=false&delayms=3000" frameborder="0" width="960" height="569" allowfullscreen="true" mozallowfullscreen="true" webkitallowfullscreen="true"></iframe>

G-slides file

Cross-project Pipeline Triggering and Visualization (May 2019 - 11.10)

<iframe src="https://docs.google.com/presentation/d/e/2PACX-1vRZaaHmlJcBQS56SSS1QKyvLJQayJ31ZXqlAJMzQOMckbHt0dSj0KBN2bzb6lwny-lqfvhfhlOl7tK8H/embed?start=false&loop=false&delayms=3000" frameborder="0" width="960" height="569" allowfullscreen="true" mozallowfullscreen="true" webkitallowfullscreen="true"></iframe>

G-slides file

Secure Capabilities (March 2019 - 11.8)

<iframe src="https://docs.google.com/presentation/d/e/2PACX-1vTmtpY92Uun3YsixCJHZu7yt69M9rFB5SuFRSglOBXRxoFNTKBdgPZpE4JBTl3LtAAX1zS4zQdBdD6Ga/eIse&loop=false&delayms=60000" frameborder="0" width="960" height="569" allowfullscreen="true" mozallowfullscreen="true" webkitallowfullscreen="true"></iframe>

G-slides file

Sales enablement recording (of shorter version)

Secure Capabilities (short, guided) (March 2019 - 11.8)

<iframe src="https://docs.google.com/presentation/d/e/2PACX-1vQF2Neh1h0vFwMapLvhprrbZZVaxbtnVvTP69xd6YNGreW5dZ43w4w5qQTmYNewml-3pViilsvblcX/embed?start=false&loop=false&delayms=60000" frameborder="0" width="960" height="569" allowfullscreen="true" mozallowfullscreen="true" webkitallowfullscreen="true"></iframe>

G-slides file

Booth Demo Auto Play link

Sales enablement recording

Auto DevOps (Oct 2018 - 11.3)

<iframe src="https://docs.google.com/presentation/d/e/2PACX-1vTNeBj0TJWca1PVwccxUTXWDWYqalyB6Q1fRYCfjtMzeK8DtpmAcG1o6ipFBi-lhYKVTAA9kYBWYKKu/embed?start=false&loop=false&delayms=600000" frameborder="0" width="960" height="569" allowfullscreen="true" mozallowfullscreen="true" webkitallowfullscreen="true"></iframe>

G-slides file

Sales enablement recording (of shorter version)

Auto DevOps (short, guided) (July 2018 - 11.0)

<iframe src="https://docs.google.com/presentation/d/e/2PACX-1vRQ4xltHhYROzgo7exF6BwR09jvPmyFzR4XvjdlpYMRqT4dctx61XCkLjR-8sq6QyOsoEFBBJjJh/embed?start=false&loop=false&delayms=600000" frameborder="0" width="960" height="569" allowfullscreen="true" mozallowfullscreen="true" webkitallowfullscreen="true"></iframe>

G-slides file

Booth Demo Auto Play link

Sales enablement recording

Auto DevOps - Setup (GKE) (Oct 2018 - 11.3)

<iframe src="https://docs.google.com/presentation/d/e/2PACX-1vQIPrinsvTG1s6ppnUWqSY-fpHnxe6oAwM7g91uBI8Mx3EYQhaeJlKUF9c3Gtaglhzwg-8dJllrNgw/embed?start=false&loop=false&delayms=600000" frameborder="0" width="960" height="569" allowfullscreen="true" mozallowfullscreen="true" webkitallowfullscreen="true"></iframe>

G-slides file

Auto DevOps - Setup (EKS) (Oct 2018 - 11.3)

<iframe src="https://docs.google.com/presentation/d/e/2PACX-1vRjhO2VvKfgWbObwi5APm18gxQrzQk5vvAARD-4vLQeT0NbkrSuP3t4sTVyIRYZOD6kiNLr37HmtHA/embed?start=false&loop=false&delayms=60000" frameborder="0" width="960" height="569" allowfullscreen="true" mozallowfullscreen="true" webkitallowfullscreen="true"></iframe>

G-slides file

Live (instructions)

Planning to Monitoring (formerly i2p)

Planning to Monitoring (formerly i2p)

Highlights GitLab's single platform for the complete DevOps lifecycle, from idea to production, through issues, planning, merge request, CI/CD, and monitoring.

CI/CD Deep Dive

CI/CD Deep Dive

Provides a more in-depth look at GitLab CI/CD pipelines.

Integration Demos

Integration Demos

Demonstrations which highlight integrations between GitLab and common tools such as Jira issues and Jenkins pipelines.

Conference Booth Setup

We've started using iPad's to run click-through demos at conferences, and also we now have a main screen video to loop that shows our key slides and product clips. You can find details about both of these on the conference booth setup page.

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/demo/acs-setup.md

Video

The video below shows how to install GitLab EE onto Azure Container Service. For the DevOps lifecycle, please refer to the sales demo.

```
<figure class="videocontainer">
  <iframe src="https://www.youtube.com/embed/MoLKfQxRaGE" frameborder="0"
allowfullscreen="true"> </iframe>
</iframe>
</figure>
```

Preparation

- > - You need an Azure account. A free trial account will suffice. Ensure you are logged in.
- > - Login to Azure Portal.
- > - This script assumes the tanuki.website domain, but you should either:
 - > - Pick the least-recently used domain from the Google Doc. (Let's Encrypt limits SSL cert creation on a weekly basis, so rotating usage helps reduce hitting the limits), or
 - > - Buy a new domain for your demo and substitute throughout the script.
 - > - Create DNS Zone to let Google manage DNS for you.
 - > - Click Registrar Setup to see what name servers to use.

- > - Disable desktop notifications (on a Mac, top-right corner, option click).
- > - Open up new browser window so the audience doesn't see all your other open tabs.
- > - Share just the web browser window so the audience isn't distracted by notes or other windows.
- > - [Optional] Go to 'Displays' settings, Resolution: Scaled, Larger text.
- > - [Optional] Open this page on an iPad that has screen lock disabled.
- >
- > CLI setup
- >
- > - On macOS, install brew for all the things
- > - `ruby -e "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/master/install)"`
- > - You need to have the Azure CLI installed. e.g.
- > - `curl -L https://aka.ms/InstallAzureCli | bash`
- > - `sudo az acs kubernetes install-cli`
- > - Install helm
- > - `brew install kubernetes-helm`
- > - `az login`

Set up a container scheduler cluster

We're going to install everything from scratch and we'll start by creating a new container cluster. Today I'm going to use Azure Container Service, which is a Kubernetes platform hosted by Microsoft. We're going to use the CLI and I'm already logged in, so the first thing is to create a group to house the cluster. I'll create it in the northcentralus zone. Then we create the cluster and name it tanukiWebsite, based on the domain name I'll use. I'll only use 1 node so it fits in a free trial account, but you can bump up the agent-count if desired.

- > - `az group create -n GitLabDemos -l northcentralus` (Change group name to something appropriate for you)
- > - `az acs create -n tanukiWebsite -d tanuki -g GitLabDemos --generate-ssh-keys --orchestrator-type kubernetes --agent-count=1 --agent-vm-size StandardDS2v2` (Change cluster name to reflect your base domain name)

Now that we have created the cluster, we can go back and check on our cluster.

- > - Navigate to Container services.
- > - Confirm it's up.

Good, our cluster is ready for us to use. Let's connect to it.

- > - `az acs kubernetes get-credentials --name tanukiWebsite -g GitLabDemos`
- > - `az acs kubernetes browse --name tanukiWebsite -g GitLabDemos`

Set up GitLab itself

Now that we have our cluster configured, we're ready to install GitLab. To do this, we'll need the base domain name, and an email address to use with Let's Encrypt. Then we use helm to install all the necessary components.

- > - helm init
- > - helm repo add gitlab https://charts.gitlab.io
- > - helm upgrade -i tanuki --namespace gitlab --set baseDomain=tanuki.website,legoEmail=you@gitlab.com,provider=acs,redisDedicatedStorage=false,postgresDedicatedStorage=false gitlab/gitlab-omnibus (Replacing baseDomain, and legoEmail as appropriate.)
- > - Note, if you're using a non-trial account, we recommend dropping redisDedicatedStorage=false,postgresDedicatedStorage=false which will isolate your database disks.
- >
- > Alternate instructions for GitLab EE
- >
- > - Go to /free-trial/ and enter in your info to request a trial license for GitLab EE
- > - Wait for email
- > - Download license to ~/.gitlab-license
- > - Install helm chart, adding the gitlab and gitlabEELicense options:

console

```
export LICENSE= cat ~/.GitLab.gitlab-license
helm upgrade -i tanuki --namespace gitlab --set baseDomain=tanuki.website,legoEmail=you@gitlab.com,provider=acs,redisDedicatedStorage=false,postgresDedicatedStorage=false,gitlab=ee,gitlabEELicense=$LICENSE gitlab/gitlab-omnibus
```

GitLab is now deploying and will take a while. The first thing is to wait until the load balancer spins up so we can grab the IP address assigned and configure our DNS with it.

- > - kubectl get svc -w --namespace nginx-ingress nginx
- > - Wait until the output changes with an EXTERNAL-IP
- > - Configure DNS to point a wildcard to that IP. Similar to:
- > - .tanuki.website 300 IN A 1.1.1.1 (Note you want a short lifespan for this record so you're not waiting a day every time it changes.)
- > If using Microsoft Azure DNS zones:
- > - Click DNS zones from the menu on the left.
- > - Click on the Zone that has the name of the domain to be used for the demo. (e.g. tanuki.website) or click + Add, give it a name, and a Resource group.
- > - Click on the + Record Set button.
- > - Set the Name to .
- > - Set the IP Address to the External-IP from the nginx service.
- > - Set TTL to 5 and TTL unit to Minutes.
- > - Click the OK button at the bottom of the page.

Now let's check if our gitlab service is up, and wait for it if not.

- > - kubectl get deployment -w gitlab --namespace gitlab
- > - Wait until Available shows 1.

Optional filler

We'll watch the Kubernetes dashboard for all items to have a green checkmark

showing that they have completed. This process can take a few minutes as ACS allocates resources and starts up the various containers. You can see here there are several containers. The main GitLab container has the Rails app, but also Mattermost for Chat, the integrated Docker Registry, and Prometheus for monitoring. Then there's separate containers for Postgres and Redis and the autoscaling GitLab Runner for CI and CD. This is everything you need for the application development lifecycle on Kubernetes.

While we're waiting: In the rest of the demo, I'll take you through everything you need to take ideas to production, including chat with Mattermost, issues and issue tracking, planning with issue boards, coding with terminal access, committing with git version control, merge requests for code review, testing with continuous integration, getting peer reviews with live review apps, continuous delivery to staging, deploying to production directly from chat, cycle analytics to measure how fast you're going from planning to monitoring, and lastly, Prometheus monitoring of your GitLab instance. With GitLab, everything is integrated out of the box.

What takes 10 minutes in this demo would take days if you're not using GitLab and have to integrate different tools. Not only is GitLab faster to set up, but it is also more convenient to have everything in one interface. Developers want to work on creating a great product, not on learning and maintaining the integrations between their tools.

If there is more time talk about what a review app is and what cycle analytics are.

> - Wait for gitlab pod to go to green or deployment to show available

Looks like our deployment is finished. Let's check out GitLab...

> - Go to gitlab.tanuki.website (Adjusting the URL to the domain you used for this demo)

Boom, we've got a shiny new GitLab installation!

Set root password

Before we get too carried away, we need to secure the root account with a new password.

> - Set password for root user (You don't need to actually log in as root, but you can)

Cleanup

> - Delete the cluster

Troubleshooting

Various failures block Let's Encrypt, and thus GitLab

There are several scenarios which can cause deployment failures due to issues

surrounding the kube-lego-nginx and the Let's Encrypt (LE) ACME process. The easiest way to find these errors is checking the logs of the kube-lego-nginx service in the kube-lego namespace of the dashboard for your Kubernetes cluster.

1. Let's Encrypt top-level domain request rate limit exceeded

The failure mode here is the most vague from the logs, however it occurs when you have exceeded the number of certificate or renewal requests allowed for a single TLD. Please see their documentation regarding this.

1. Unresolvable DNS

If your DNS records are not correctly configured, the Let's Encrypt servers may not be able to resolve your domain when the ACME requests are filed against it. Let's Encrypt performs a reachability test that depends on valid, resolvable Fully-Qualified Domain Names. You must confirm that your entry DNS is functional, and has propagated. You can do this by using an external host (anywhere not directly querying your primary DNS where this record is present) to ping test.my.tld where my.tld is the domain name you are using. Because you should have configured a wildcard record (.my.tld), test.my.tld should resolve to that address.

1. Host not responding (reachability)

This has been observed as a failure of the LoadBalancer to be properly connected to the reserved static external IP address. There are a few methods of failure here, but the primary cases are:

- Unable to assign due to prior assignment.

Either an existing use, or a failure to fully remove the prior deployment. This has been seen in both scenarios by GitLab personnel. If you are re-creating a previous deployment, you need to wait a short period and/or confirm that the previously used GCP Networking LoadBalancer has been removed. You can manually remove these if you do not wish to wait for GCP to catch up with the de-provisioning.

- Unable to assign due to incorrect region.

If you inadvertently create a GKE Kubernetes cluster in a region that is not the same as the static IP address you are attempting to use, your deployment will fail to attach to that IP address, and result in the inability to listen and respond to requests.

General notes

Logs

- You can find logs for each pod in the Kubernetes Dashboard
 - Select Namespace you want to see logs for
 - Navigate to Pods
 - Select Pod you want to see logs for
 - Click on View logs
- You can check logs from CLI using kubectl as well

```
console
kubectl get namespaces
kubectl get pods --namespace=<NAMESPACE>
kubectl logs <POD> --namespace=<NAMESPACE>
```

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/demo/cicd-deep.md

CI/CD Deep Dive

Let's take a more in-depth look at GitLab CI/CD pipelines.

```
<figure class="videocontainer">
  <iframe src="https://www.youtube.com/embed/pBe4t1CD8Fc?start=1885" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
```

Configuration and Integration

Let's start at the top with configuring CI and setting up any required integrations.

Connecting to a Repository (Mirroring)

Now, we've already installed GitLab earlier so we will continue on to connecting our source code repository. The vast majority of our customers utilize our built-in Git repository functionality, due to it's great feature set and seamless integration with CI/CD. However for those would like to utilize a different SCM tool, you can connect any Git based repository by simply configuring mirroring.

> Project Settings > Repository > Pull from a remote repository

We'll get started with our built-in repository and a simple Java app which is composed to two parts. A Java library and a front-end service which depends on that library to respond to users with a simple greeting.

Integration with Agile Planning Tools (JIRA & Slack)

Similar to the repository, GitLab also includes a powerful set of planning tools as you saw earlier. However we also have great integration options here as well, for example tools like JIRA and Slack. Let's see how these are configured now.

Configuring JIRA takes just a minute or two. We've provided our server's URL, credentials to authenticate, and then our project information: the key and a transition so we can close issues. That's it!

> Project Settings > Integrations > JIRA

Enabling Slack notification and ChatOps commands are similarly easy. For notifications we

simply choose what events we want to push to our channel, and then provide a webhook for our Slack server. In this case we are pushing Pipeline events, for both successful and failed pipelines on all branches.

> Project Settings > Integrations > Slack Notifications

Working with `.gitlab-ci.yml` (Versioned, Ways to edit)

> Repository

Now, setting up downstream tool integration is accomplished by working with our pipeline definition file, `.gitlab-ci.yml`, which specifies the stages, jobs, and actions we want to perform.

Before we do that though, let's just talk briefly about the various options for working with this file. As you can see, this file is checked in to our repository, which provides a number of benefits:

- The file is versioned which means pipeline changes can be tested in branches supporting any changes in your app code. Similarly if you need to go back to an old version, perhaps to ship a security release, the associated pipeline will be exactly how you left it for that particular release.
- It also means that there are a large number of ways to work with this file. Nearly all IDE's have direct integration with Git, so you can use your favorite editor. The classic CLI is of course always possible as well, as well as our integrated web based editor that you can see here.

Let's now take a closer look at this file, and how you can define the pipeline and integrate with a wide variety of tools.

Settings in `.gitlab-ci.yml` (Self Service, Bash script, Templates)

> Click on the `.gitlab-ci.yml` file to view

At the top of the file we have defined a few global defaults:

- A docker image to run our commands within, in this case the official Maven image
- A few environment variables
- Cache settings, which allow folders to be persisted between jobs to increase performance

Next we start to define our stages and jobs. In GitLab CI, a Pipeline is made up of a series of stages. Each stage then contains one or more jobs. There are no limitations on how many jobs a single stage can have.

The `stages` keyword defines the order the stages should execute in. So our flow will be: build, test, generate docs, release, and then deploy.

Build Stage (Maven, Bash scripting)

We then start to define our jobs, or the actions we want to perform.

Our first job is to build our library, which we utilize Maven for. You can see here we simply invoke Maven, as we would as if we were running it on our machines. That is because the script you see here, is actually just Bash script. This means provides a great amount of flexibility, because you can now automate anything you would normally do on your machine.

Test Stage (2 jobs, code coverage output, artifacts)

After we built our library, we are now going to test it.

Our test stage includes two jobs: unit tests and static analysis with codeclimate. We are going to leverage Maven again to run our tests, but we'll also include jacoco to generate code coverage reports. We then output the coverage percentage into the build log. The last step in this job is to take our code coverage reports and persist them with GitLab's integrated artifact repository. We do this by simply specifying the folders we want to save.

While that is happening, we will also be running static analysis with codeclimate. Here we override the default image, to utilize the official Docker image. We use that to then execute run Docker-in-Docker, to execute the codeclimate image to analyze our source. Once that is done, we retrieve our JSON report and persist it as an artifact.

Doc Stage (Generate, Artifacts)

Here, we are simply generating our javadocs and again retaining them as an artifact.

Release Stage (Protected Variable)

Now that we have tested our library, we are ready to release it. We are going to use Maven again to publish our library out to our Packagecloud server. Now you are paying really close attention, you will see we are using a variable we did not define above. That is because this is a credential token, which should not be checked into the repository. Instead, we add this as a Protected Variable in our project settings, which only administrators can view.

Deploy Stage (Pages, Cross-Project Pipelines)

Finally we have our last stage, deploy, which includes two jobs.

Our Pages job is slightly unique, in that this is a special job that works in tandem with GitLab Pages, our static site hosting feature. With Pages, deploying a static site is as easy as creating an artifact. We do that here by specifying we will utilize the artifacts of two of our jobs: unit tests and javadocs. This job then copies those into a single directory structure and persists it as an artifact. GitLab Pages will then take this and deploy out to the integrated hosting service, providing an easy and automated way of posting in our case, our code coverage and documentation.

Now for our last job, recall that our Java app was made up of two components: this library and a front-end service which used it. At this point, we've confirmed that all of our tests pass which is a good start. But what about downstream projects that utilize this? Well this job kicks off what we refer to as a cross-project pipeline. We take a stock alpine image, install curl, then use it to trigger the webhook to start a new front-end service pipeline. Confirming downstream projects are not negatively affected by upstream changes is as easy as a few lines

of YML.

Wrap up (Self-service: no plugins, Flexible: Bash script, Templates)

As you can see, this supports our larger goals of GitLab: scalability, flexibility, and self-service:

- A developer can integrate with any tool they need, without having to worry about installing plugins or involving the administrators. They can simply provide the required container or VM to run the script in, or install their own runner on a machine with the associated requirements.
- Integrating with static analysis and unit testing frameworks is just a lines of code.
- Moving into the Part 2, you can see integrating with build automation tools like Maven is similarly straight forward and easy.
- We also have a collection of templates which can be used to help users get started.

Deep Tool Integration & Intelligence (Code Climate, Code Coverage, JIRA Issue)

Now we already showed how easy it is to execute unit tests and static analysis, but our integration goes a lot deeper. I have a pending merge request ready for me to review, so lets take a look.

> Open pending merge request

As a code reviewer, my job is really important: to ensure that only high quality code that meets our standards is merged. To assist in manual review, we often include automated tools to like static analysis and code coverage. However these reports can often be lengthy and tedious to review, leading to decreased usage. With GitLab's holistic vision, we have a unique opportunity so solve these challenges.

So it looks like one of our developers forgot to clean up a stray FIXME comment that was already addressed. Our pipeline succeeded, so that is promising. We built our code, ran our unit tests and static analysis, and generated our javadocs. That's all pretty straight forward, but as you can see our integration goes further with additional intelligence.

Here we've extracted the code coverage from the unit testing job and present it directly on the interface. We've also done analysis of the codeclimate report. By comparing the results from this branch to the branch we are looking to merge into, we can provide the reviewer with the delta in code quality if this were to be accepted. Instead of sifting through a bunch of results, I'm presented with what is relevant to this change.

Of course down below we can review and collaborate on the actual code changes. But this all looks good, so let's go ahead and merge.

> Merge

Now similar to what you saw earlier with our integrated issue feature, we will go ahead and close the associated JIRA issue as well. Let's take a quick look at this issue.

> Show it is already linked, and now closed. Close tab and return to MR.

Executing Build Automation

By committing a change to master we now have a new pipeline running, let's take a look to see how it executes and the notifications we receive.

> Command-Click on Pipelines to open a new tab, then click on the pipeline we just started on Master to see the overview.

We can now see a real-time overview of our pipeline as it progresses. Our build stage just completed, and you can now see our two test stages executing in parallel. Let's take a closer look at our unit test job.

Unit Tests Job (Maven, Code Coverage, Artifacts)

> Click on unit tests job

As before, we get a live look at the build log as it is being executed by our Runner. You can see Maven is executing, and finally our tests have passed. Here is the code coverage output which GitLab is parsing, and our artifacts. On the right hand side, you can see we are presenting the code coverage and also offer a way to browse the artifacts. They can also be accessed in the future by other jobs.

Now let's go take a look at our codequality job.

Code Climate Job (Docker, Artifact)

> Click on codeclimate job from right pane

In this job, you can see us downloading the codeclimate image and executing it. We then collect the output and persist as an artifact. It's that easy.

Going back to pipeline overview, we can check back in on our progress.

> Click on Pipeline at the top of the page to go back to overview

Our release job is now executing, let's see how that is progressing.

Release Job (Protected Variable, Maven deploy)

We're now ready to publish our library to our packagecloud server, utilizing Maven. Remember we had utilized a protected variable for this, to ensure the secure token was not committed into the repository.

> Scroll up in build log to show the environment variable functioning

And that's it, our library is now available for others to consume.

> Go back to pipeline overview

Pages Job (as necessary to kill time for success notifications)

In our Pages job, we are leveraging the artifacts we collected in our earlier stages. Here, we simply copy them into a single directory structure, and persist them in a new artifact. Our GitLab Pages service will take over from here to copy our code coverage and documentation to our static site. Note this works with any static site generator as well.

Notifications - Success: Browser, Slack, Favicon, Email (off by default)

As you can see from our notifications, our pipeline has succeeded. In the event you didn't catch them all during that flurry, let's review them quickly:

- GitLab itself will alert you via browser notifications when a pipeline succeeds or fails.
- Since we configured Slack notifications, we received a notification there as well. Let's take a look.

> Switch to Slack tab, show notification

- GitLab also updates the favicon for both our MR and pipeline views, indicating we were successful.
- Last we could also have received an email as well, however by default we do not send them for successful pipelines.

> Go back to pipeline overview

Cross Project Pipelines (Environment Screen, Registry)

Let's check in and see how our Cross Project Pipeline is doing, for our front-end service.

> Click on downstream stage

Here you can see a relatively simple pipeline:

- We will build project
- Execute our tests
- Package up our JAR file
- Build our Docker container
- Deploy to Production

> Click on Production

Great, it looks like our deploy has finished and our environment is available. You can always get more information on an environment by clicking on it, where we will show you the deploy history as well as options for monitoring and troubleshooting.

We can also get a closer look at our integrated container registry.

> Click on the Registry tab

Here we can a view of all the containers we currently have pushed to the registry, and their associated tags. If we'd like, we can manually remove some if we don't need them anymore. We also provide a tool to manage your containers in bulk, as well.

Aside from an integrated UI, one of the key benefits of integrating the registry is the unified authentication architecture. Instead of having to manage credentials and security on your own, GitLab makes it extremely simple. We provide a simple environment variable to access the registry which is valid for as long as the job is executing.

Pipeline Wrap-up (flexible, self-service)

So that's our "Happy Path". With just a few lines of YAML we accomplished:

- Integration with Maven, Unit Tests, Code Coverage, and Code Climate
- Generation of Javadocs
- Releasing our library to Package Cloud
- Publishing the latest documentation to our team's site
- And triggering downstream project pipelines to ensure there are no negative impacts

And we did that all without involving a single administrator or opening a ticket. Completely self-service by a developer.

Negative Path

But you know what? I think we can make some improvements to library.

> Go back to `dep.java` and edit the `bye` text.

This library, as you can see, greets and wishes farewell to our users. Let's make it a little more polite though, by saying "Goodbye".

We'll commit this to a new branch, and create a new Merge Request.

> Commit to new branch, create new Merge Request

Failed Merge Request (MWPS, GitLab, Slack, Favicon, Email)

Since we've now created a new branch, we have a new Pipeline running. Now if I am the reviewer and already checked out the changes, I can simply click on our Auto-merge button. This will automatically merge the issue as long as all checks pass, including a green pipeline. However if it fails and the developer needs to make a further change, it will of course have to get re-reviewed. This is a great way to save some time for your reviewers, so they aren't waiting until a pipeline complete before moving on.

Now we're going ahead and building the new version of our library, and next up we'll be running our tests. Let's see how our unit tests are going.

> Command-Click on unit tests job

Uh oh, as you can tell from the notifications it looks like our tests failed. In this case, we changed the message but forgot to update our tests.

Let's quickly review the notifications we received:

- GitLab itself sent a browser notification, indicating the failed pipeline
- With our Slack integration, we received an alert there as well. (Show Slack)
- If you take a look, our favicons also updated to indicate an errors
- And finally since this was a failed pipeline, we received an email alert as well

> Show email tab

> Return to the job log for the unit tests

Failed Merge Follow up (New Issue, New Todo)

As you can see we have a wide array of ways to inform users that their pipeline failed. But what is at least as important as notifications, is follow up.

Looking back at our failed job we can see a new button has appeared, to easily create an issue for this problem. Clicking it will open an issue with some helpful information already filled out, like the build log.

Looking at the top right corner of our screen, you may also have noticed we have a new To Do. To dos are essentially an automatic built in tool,